Introduction to Claude Code

AI workflows for research — from first prompt to publication-grade analysis

EARTHLAB 2026

Francesco Muschitiello

Department of Geography, University of Cambridge

THE PREMISE

What is Claude Code?

Not a chatbot — an agentic environment that reads your files, runs commands, and iterates until the job is done.

Reads your files

Understands your project structure, code, and data without being told. Point it at a folder and it it orients itself.

Runs commands

Executes code, installs packages, runs tests, inspects outputs — on your machine, with your approval.

Iterates

Plans, implements, hits an error, reads the traceback, fixes it, retries — without you babysitting each step.

```
> Train the CNN for 5 epochs on the 2015–2020 images and report test accuracy per class.
  ● Bash(python train.py)  → RuntimeError: expected 1 channel, got 3
  The loader is missing the grayscale transform. Fixing and retrying.
  ● Edit(loader.py)  ● Bash(python train.py)  → accuracy 0.91 ✓
```

You describe the outcome. Claude figures out how to get there — that error→fix→retry loop above happened with zero intervention.

ONE APP, THREE TABS

The Claude ecosystem

The Claude desktop app puts three ways of working side by side. Same models underneath — different levels of autonomy.

Chat

Conversation, document analysis, image understanding. No file access.

Ideal for: summarising PDFs, brainstorming, translating, turning a brain-dump into a structured prompt.

Code

THIS WEEK

Interactive agent with direct access to your local files. You review and approve every change.

Ideal for: analyses, pipelines, figures, debugging, git workflows — your day-to-day research.

Cowork

Autonomous background agent in its own cloud workspace. Runs while you do other things.

Ideal for: document creation, file organisation, research synthesis, scheduled recurring tasks.

We focus on the Code tab of the desktop app. Everything also works in the terminal CLI — if you're feeling brave, that's your weapon of choice.

WHY THIS MATTERS NOW

The frontier, as of this week

Opus 4.8 (May 28) and Fable 5 (June 9) — the first Mythos-class model researchers can actually use.

Months of engineering → one day

Stripe ran Fable 5 on a codebase-wide migration in a 50M-line Ruby codebase — done in a day, vs an estimated 2+ months for a team by hand.

Novel science, end to end

Mythos 5 assembled single-cell data across 138 species and trained a custom ML model that beat a model published in Science — at 1/100th the size.

Hypotheses worth testing

Its proposed mechanism for an E. coli protein was corroborated by an independent lab. Anthropic's scientists preferred its hypotheses ~80% of the time.

Real open-source, one afternoon

Simon Willison shipped a release of his LLM library “almost entirely written by Fable” — API design, tests and docs he called several days' worth of work.

Same models, same tools you'll use this week — pointed at your data, your pipelines, your papers.

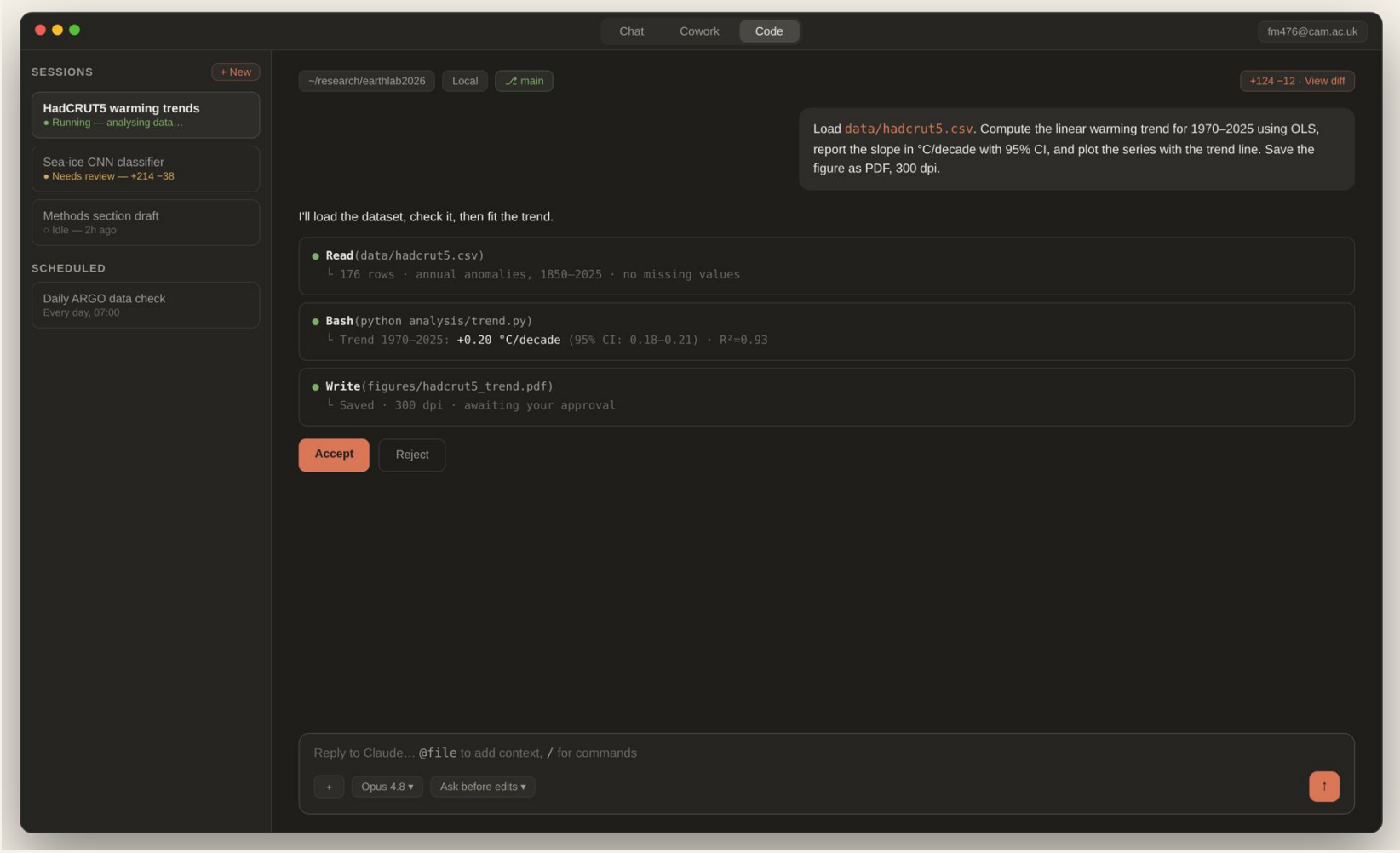
PART 1

Getting started

The desktop app, models, and how to talk to an agent

YOUR HOME BASE

The desktop app at a glance



Sessions sidebar

Each task gets its own session with its own context. Run several in parallel.

The agentic loop

Watch Claude read, run, and edit — every tool call is visible, nothing is hidden.

You stay in control

Ask-permissions mode: Claude proposes, you accept or reject each change.

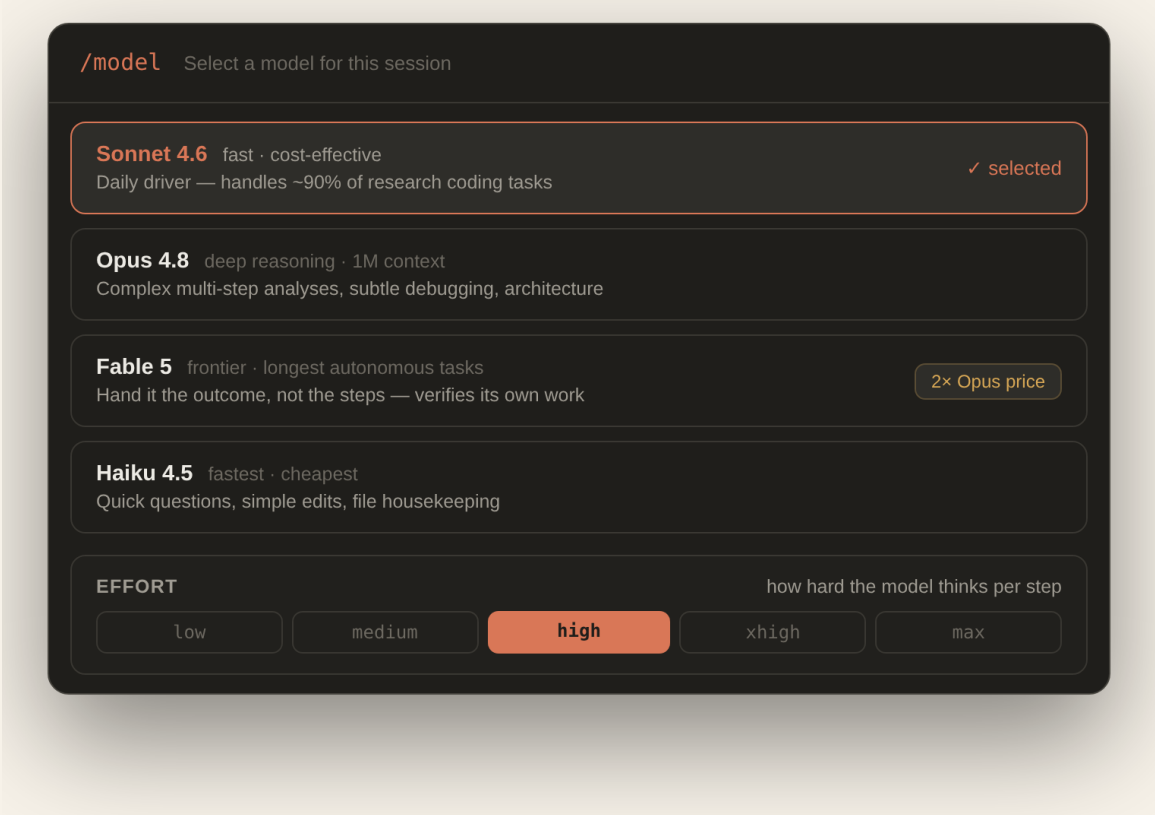
Model & permissions

Switch models and permission modes from the prompt box, mid-session.

MODELS

Choosing the right model

Sonnet is your default. Opus is your specialist. Fable is your heavy machinery.



Sonnet 4.6 — the default

Fast and cost-effective; handles ~90% of research coding: data wrangling, figures, debugging, tests.

Opus 4.8 — the specialist

Deeper reasoning, 1M-token context. Use for multi-file refactors, subtle statistical bugs, unfamiliar codebases.

Fable 5 — the frontier (2× Opus price)

Mythos-class. Hand it the outcome, not the steps: it plans, sustains long sessions, and verifies its own work. Reserve for your hardest problems.

/model switches any time · /effort tunes thinking depth · escalate only when reasoning — not effort — is the bottleneck.

PROMPTING · 1

Be specific

The more precise the request, the fewer correction rounds.

VAGUE

> analyze this climate data

VAGUE

> Make a figure of the results

SPECIFIC

> Load hadcrut5.csv. Compute the linear warming trend for 1970–2025 using OLS. Report the slope in °C/decade with 95% CI. Plot the series with the trend line.

SPECIFIC

> Two-panel figure: top — global, NH, SH anomalies (1850–2025); bottom — decadal warming rates as bars. Colorblind-friendly palette. Save as PDF, 300 dpi.

Brain-dump → structured prompt: the Chat tab is your friend — paste your messy thoughts, ask it to draft the precise prompt.

Prompt like a collaborator

A 30-second setup conversation saves 30 minutes of corrections.

> I have a CSV of HadCRUT5 annual temperature anomalies. I want to check for data quality issues before running trend analysis.
>
> Before you start, ask me any questions you need answered to do this well. What checks would you recommend, and why?

Treat Claude as a colleague

Give context, explain the goal, share what you already know. The more it understands, the better it performs.

Ask Claude to ask questions

“Before you start, ask me anything you need.” This surfaces assumptions you didn't think to specify.

Plan together before coding

A short planning exchange catches bad approaches early — before Claude writes 200 lines you have to debug. (Plan mode formalises this — coming up.)

PROMPTING · 3

Break down large problems

Your problem-solving skills still matter — Claude amplifies them.

TOO BIG

> Build a complete pipeline that downloads sea-ice satellite data, classifies it with a CNN, and generates a report with figures.

SOLVABLE

> **Step 1:** Write just the data loader. Read images from train/, apply the grayscale transform, return a PyTorch DataLoader. Nothing else yet.
> **Step 2:** Build a simple 3-class CNN on that loader. Train 5 epochs, print accuracy.
> **Step 3:** Summary report: three confusion matrices + a paragraph on whether the model generalises across years.

Each step is verifiable. You inspect the output before building the next stage on top of it.

PROJECT SETUP

CLAUDE.md — your project's memory

A briefing file Claude reads at the start of every session. Run /init to generate one, then prune.

Include

Build commands Claude can't guess
conda env, package quirks, data paths
Conventions that differ from defaults
“use pathlib”, “temperatures in Kelvin”
Gotchas you keep repeating
“the .csv uses semicolons, not commas”

Exclude

Anything Claude can read from the code
imports, folder structure, function names
Standard conventions & long tutorials
link, don't paste
Frequently changing information
current results, figure numbers

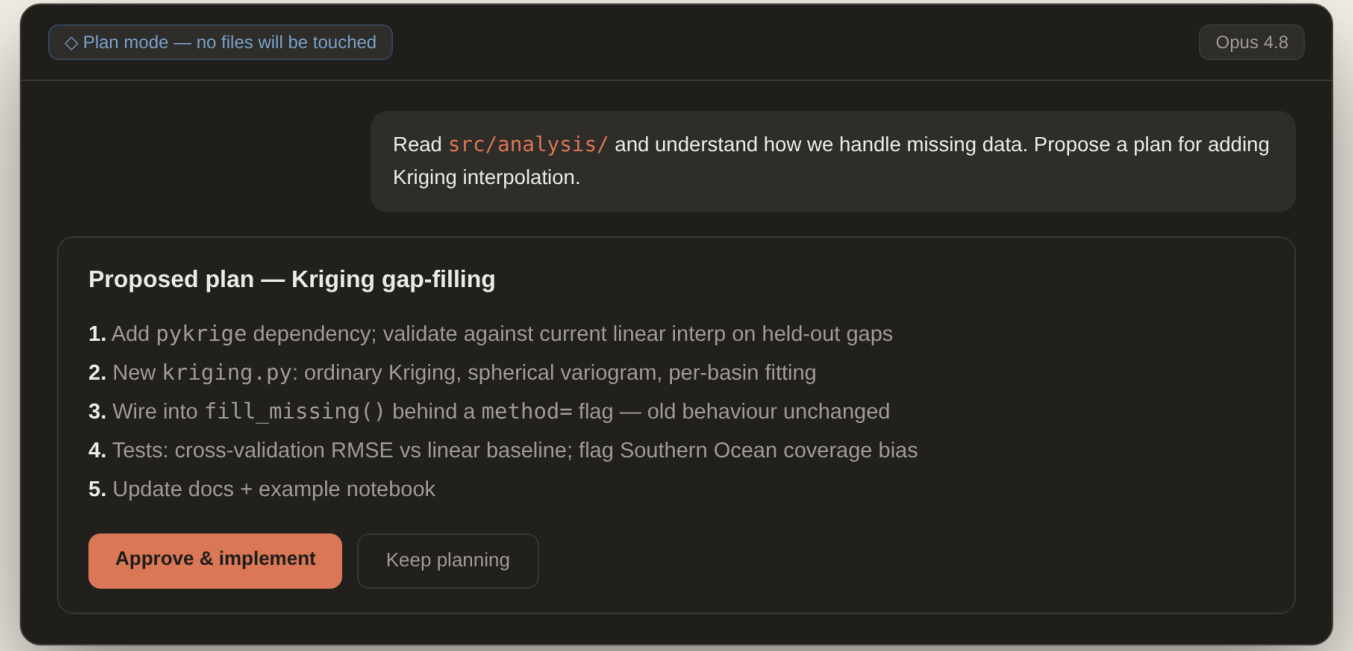
```
> /init
● Analyzing project structure…
● Generated CLAUDE.md with detected patterns
```

Keep it lean. For every line ask: would removing this cause mistakes? If not, cut it. If Claude keeps ignoring a rule, the file is too long.

PROJECT SETUP

Plan first, code second

Permission modes set how much autonomy Claude gets. Plan mode is the one to remember.



Plan mode

Claude reads and thinks but touches nothing. Iterate on the the plan until it's right, then approve.

Ask permissions (default)

Every edit shows as a diff with Accept / Reject. Best while you're learning to calibrate trust.

Auto-accept edits

Faster iteration once trust is earned — review the full diff at the end instead of each step.

Workflow: Explore → Plan → Implement → Commit. Skip planning for small tasks; use it whenever the approach is uncertain.

PART 2

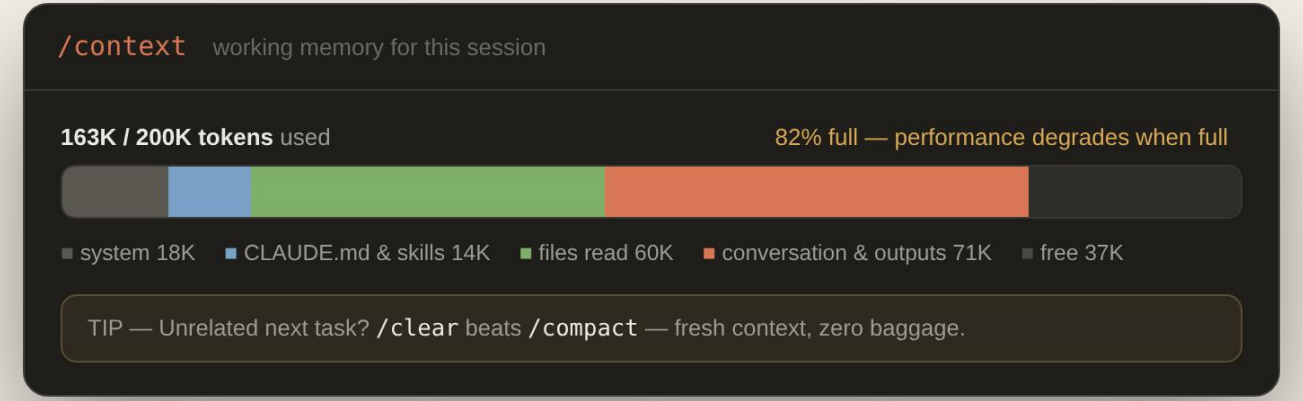
Context is like milk

Best served fresh and condensed

WORKING MEMORY

The context window

Everything Claude currently "holds in mind": instructions, files read, command outputs, the whole conversation.



Everything fills it

Every message, every file read, every command output consumes tokens.

Performance degrades as it fills

Earlier instructions fade; mistakes increase. Full ≠ fine.

Bigger isn't a fix

Opus & Fable offer 1M-token context — fresh still beats full.

When context gets full, Claude starts forgetting earlier instructions and makes more mistakes.

HABITS

Managing context

Four habits that keep sessions sharp.

/clear — between unrelated tasks

Reset completely. The single most important habit: new question, new context.

/compact — mid-task breather

Compress the conversation, keep the essentials. Add a focus: /compact focus on the API changes.

Esc / rewind — stop & undo

Interrupt mid-action; /rewind restores a previous checkpoint of code + conversation.

Handoff document — for long tasks

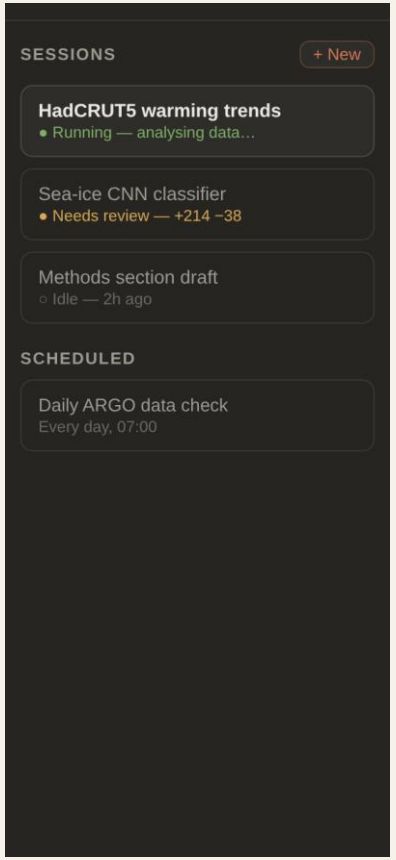
Before clearing: “Write HANDOFF.md — what you tried, what worked, what didn't.” Start fresh, point at the file.

After two failed corrections, stop. A clean session with a better prompt always beats a long session with accumulated corrections.

PARALLEL WORK

One task, one session

Instead of one long conversation that jumps topics, give each task its own session in the sidebar.



Separate contexts, zero interference

Two sessions are two Claudes. Your data-cleaning session can't pollute your writing session.

Run them in parallel

Kick off model training in one session, draft the methods text in another, monitor both from the sidebar.

Keep it manageable

3–4 active sessions max. Name them clearly. Idle sessions resume exactly where you left off.

Schedule the routine ones

Recurring checks (daily data download, weekly literature sweep) can run automatically on a schedule — even in the cloud, with your laptop closed.

Claude makes mistakes — confidently

Wrong answers arrive in exactly the same tone as right ones. Your scientific scepticism is the safety net.

Confident errors

No hesitation, no hedging — the delivery gives you zero signal about correctness. Verify the claim, not the tone.

Fabricated references

Papers, methods, and APIs that don't exist, cited with plausible authors and years. Check every reference before it enters your work.

Flawed statistical logic

Reasoning can look textbook-perfect and be subtly wrong — inappropriate tests, ignored autocorrelation, silent assumption violations.

Plausible-looking bugs

Scripts can produce sensible-seeming results for the wrong reasons — the most dangerous failure mode in research code.

VERIFICATION

Making Claude check its work

1 Write tests

Have Claude write and run tests for its own code — it's good at breaking things it just built.

2 Subagent review

A fresh context reviews independently: “use a subagent to review this analysis for errors.”

3 Verification table

“Double-check every claim and script. End with a table of what you could and couldn't verify.”

4 Audit agents on completion

Spawn independent reviewers with different expertise — statistics, data quality, code — each reporting separately.

```
> Double-check everything: every claim,
  result, and script. At the end, make
  a table of what you were able
  to verify — and what you weren't.
```

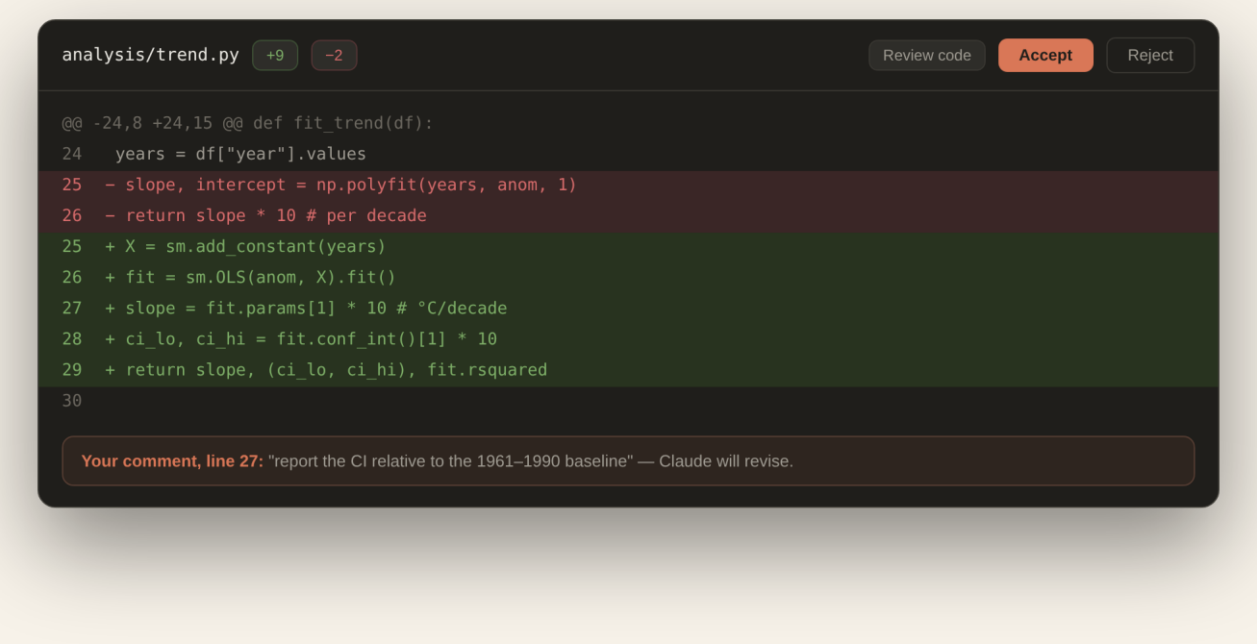
```
> When you finish, spawn three
  independent agents to audit it:
  one for statistical validity, one
  for data-quality assumptions, one
  for code errors. Report separately.
```

CODE REVIEW

Review at the right level

Move along the spectrum depending on the stakes — and comment directly on the diff.

Vibe coding	Guided	Line by line
quick prototypes, one-off scripts	*review structure, check key functions*	*verify every change — publication code*



Approach before code

If the plan is bad, the code will be too. Review the outline first.

Make Claude explain it

Walk through any block in plain English — you'll level up fast by seeing the patterns.

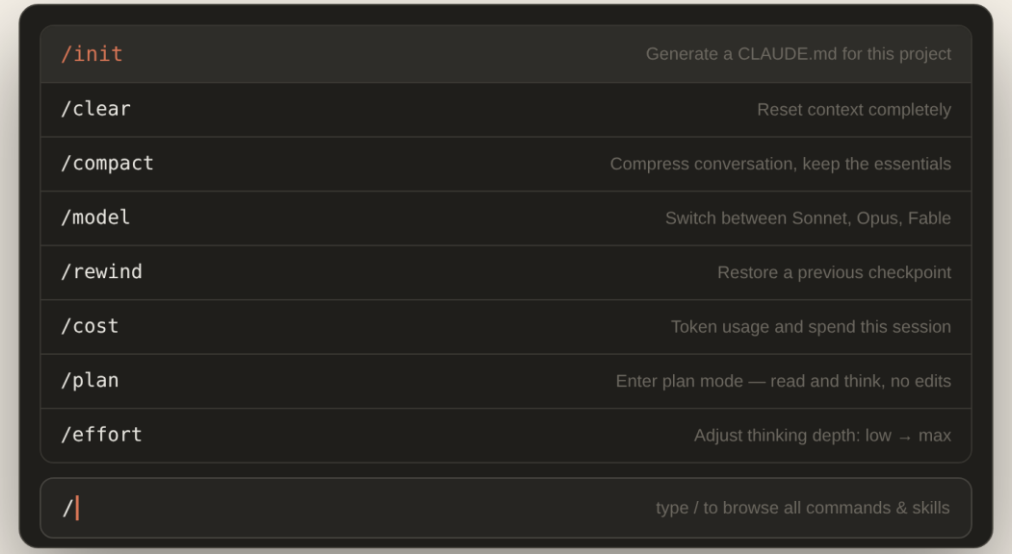
Second opinion

“Review code” makes Claude critique its own diff; a fresh session reviews with no attachment to the approach.

Shipping code you don't understand is still a cardinal sin. Review at a higher level — but review.

Essential slash commands

Type / in the prompt box to browse everything — these are the ones you'll actually use.



/init	generate CLAUDE.md for the project
/clear	reset context between tasks
/compact	compress the conversation
/model	Sonnet ↔ Opus ↔ Fable
/effort	thinking depth: low → max
/rewind	restore a previous checkpoint
/cost	token usage & spend
/plan	enter plan mode

Skills and plugins appear in the same menu — type / and your custom commands sit next to the built-ins.

THE TOOLKIT · 2

Skills — reusable domain knowledge

A folder with a SKILL.md that teaches Claude a procedure once, so you never re-explain it.

```
# .claude/skills/climate-audit/SKILL.md
---
name: climate-audit
description: Audit climate datasets for errors
---
When auditing climate data:
1. Check for duplicate timestamps
2. Verify units and sign conventions
3. Plot regional breakdowns
4. Compare against known benchmarks
```

Loads when relevant

Claude reads the description and pulls the skill in when the task matches — or invoke it manually: /climate-audit.

Your lab's tribal knowledge

QC procedures, plotting styles, naming conventions — written once, applied every session, shared via git.

Skills vs CLAUDE.md

CLAUDE.md is always loaded — keep it lean. Skills load on demand — put the detailed procedures there.

We'll build this exact skill together in the practical.

THE TOOLKIT · 3

Essential tools & skills

A non-exhaustive list of my favourites — install once, then invoke from the / menu like any built-in.

skill-creator	/skill-creator	Build new skills from scratch, edit existing ones, run evals. The best way to learn what a skill is — make one.
caveman	/caveman full	Token compression (~75% cut) with accuracy kept. Three levels: lite / full / ultra. Stretches your usage limits.
deep-research	/deep-research	Fan-out web search, fetch sources, adversarial fact-check, synthesise with citations. Ideal for focused lit reviews.
double-check	/double-check	Forces evidence — commands actually run, with results — before Claude may claim a task is done.
workflows	built-in tool	Deterministic multi-agent orchestration: fan out N agents, pipeline stages, adversarial verification.
ruflo	/ruflo	Agent-swarm orchestration: spawn agents, route tasks, hive-mind coordination, autopilot mode.
gsd:* suite	/gsd:new-project	Goal → phased project execution with plans, exit criteria, and verification gates that survive across sessions.

Where to find more: the + → Plugins menu in the desktop app, plus claudeskills.info — and the resources slide at the end.

Agents & subagents

A subagent is a second Claude spawned for a subtask — fresh context, no shared history; it reports back and exits.

```
# .claude/agents/reviewer.md
---
name: reviewer
tools: Read, Grep, Bash
model: opus
---
Review code for statistical errors.
Check assumptions. Flag unverified claims.
```

```
> Use subagents to investigate how our
  pipeline handles missing values, and
  report back findings.
```

Clean context

The subagent reads 50 files so your main session doesn't have to hold them — only the findings come back.

Parallel work

Several subagents at once: one per dataset, one per hypothesis. Opus 4.8 can orchestrate hundreds in dynamic workflows.

Independent review

A reviewer with no memory of writing the code has no attachment to its mistakes — specialist roles, honest audits.

Beyond code — the universal interface

Whatever you want done on your computer, Claude Code is the first place to try.

```
> Analyze the NetCDF files in my data/ folder
> Convert this .mat file to a clean DataFrame
> Find what's using all my disk space
> Create a publication figure from this CSV
> Transcribe these audio field notes
> Rename 400 files to ISO dates, by content
```

Connects to your tools

GitHub, Slack, reference managers, lab drives — via connectors (MCP).

Watches your PRs

Opens pull requests, monitors CI, fixes failures, merges when green.

Runs on a schedule

Daily analyses and briefings run in the cloud — laptop closed.

If a task involves files and a computer, it's probably automatable — ask before you assume it isn't.

Writing & documentation

Claude reads your actual code and data — drafts reflect what you did, not generic boilerplate.

```
> I have a notebook with my sea-ice
  analysis (analysis_v3.ipynb).
  Draft a methods section for my paper.

  Include: data sources & periods,
  statistical methods, assumptions
  and limitations.

  Third person, past tense, Nature
  Climate Change style. Under 500 words.
```

1 Give context

Point at the notebook, data, outline — the more it knows, the better the draft.

2 Specify style & format

Journal, tense, word limit, audience. Don't make Claude guess.

3 Edit collaboratively

“Move this paragraph”, “emphasise uncertainty”, “add the caveat about coverage bias”.

4 Verify every citation

Generated references must be checked against the actual literature. Always.

Be braver in the unknown

Even in unfamiliar territory, you can do far more with Claude Code than you think.

Never used PyTorch?

Ask Claude to build the model and explain each step as it goes — you learn while it builds.

Unfamiliar codebase?

Ask for an architecture tour before touching anything — “explain how data flows through this repo.”

New statistics?

Ask Claude to implement the method and walk you through the maths with intuitive examples.

```
> I've never used MCMC before. Walk me through fitting a model to this
  temperature series. Explain every step like I'm a smart teenager, and
  generate intuitive graphics as you go.
```

The unfamiliar is exactly where the leverage is highest — a patient expert who never tires of “why?”

FIELD GUIDE

Eight common mistakes

1

No CLAUDE.md

Claude rediscovers your project every single session.

2

Frontier models for everything

Sonnet handles ~90% of tasks at a fraction of the cost.

3

Vague prompts

“Fix the bug” → 5–8 correction rounds instead of 1–2.

4

One giant prompt

Partial implementations, lost context, undebuggable output.

5

Never clearing context

Stale sessions degrade quality — /clear between tasks.

6

Ignoring usage

/cost and the usage panel exist; surprise limits hurt.

7

Shipping unreviewed code

If you can't explain it, don't publish on top of it.

8

Using it as a search engine

Agentic sessions are for agentic work — Chat is for trivia.

WORKGROUP

Explore → Audit → Build

Hands-on: HadCRUT5 & ERSSTv5, in pairs, in the desktop app

WORKGROUP · PHASE 1

Explore

Open the Code tab in your earthlab2026 folder. The data is in data/hadcrut5.csv.

> Load hadcrut5.csv. Compute the global warming trend for 1970–present. Is the rate accelerating? Test whether NH and SH trends are statistically different. Produce a publication-quality multi-panel figure and write a one-paragraph results summary for a paper.

Work in pairs

One drives, one reviews — swap halfway. Agree the prompt together before sending.

Share on Slack

Post your figure and one-paragraph summary when done.

We'll compare results across pairs — same data, same prompt… are the answers consistent?

WORKGROUP · PHASE 2

Build & re-run

1 · Start fresh

Close your session, open a new one. Clean context, no baggage from Phase 1.

2 · Create a CLAUDE.md

Use /init or chat: “temperatures are °C anomalies relative to 1961–1990”, “we plot with matplotlib”. Set your audit guardrails.

3 · Build a climate-audit skill

Checks: duplicate timestamps, monotonic time axis, hemisphere sign consistency, outlier detection, regional trend comparison.

4 · Re-run Phase 1

Same analysis, with the skill active. Share the updated results on Slack.

Compare Phase 2 with Phase 1: what changed? Which errors did the guardrails catch?

WORKGROUP · PHASE 3

Open research

Using ERSSTv5 (in the "Claude Code Practical" folder – link here), each pair takes one task and reports back to the class.

Pair 1	**Spectral analysis**	Dominant periodicities in high-latitude NH SST, with significance testing.
Pair 2	**Partition the spatial variance**	Quantify the variance of SST by grid point, with significance testing..
Pair 3	**Spatial trends, last 50 years**	Grid-point linear trends with significance maps.
Pair 4	**EOF / PCA**	Dominant modes, physical interpretation; compare 1854–1940 vs 1950–present patterns and explained variance.

KEEP GOING

Useful resources

OFFICIAL

Claude Code docs

code.claude.com/docs

Commands, features, configuration — the desktop app guide lives here too.

Best practices

code.claude.com/docs/en/best-practices

Anthropic's own guide to getting the most out of Claude Code.

Fable 5 & Mythos 5 announcement

anthropic.com/news/claude-fable-5-mythos-5

What the frontier looks like, with the research showcases from today.

45 community tips

github.com/ykdojo/claude-code-tips

Community-curated tips — several of this lecture's slides started here.

COMMUNITY — FOR RESEARCH

claude-scholar

github.com/Galaxy-Dawn/claude-scholar

Research assistant skills: ideation → coding → writing, Zotero integration, citation checks.

scientific-agent-skills

github.com/K-Dense-AI/scientific-agent-skills

140+ scientific skills & DB connectors — incl. NOAA, NASA, USGS for earth science.

superpowers

github.com/obra/superpowers

Spec-before-code, TDD, evidence-before-done — the rigour reproducible science needs.

Skill directory

claudeskills.info

Searchable index of community skills, from data QC to figure styling.